An Explainable, Data-Sufficiency-Aware Framework for Construction Schedule-Risk Scoring with Computer Vision as Supporting Evidence

Ashish Upadhyay
Dept. of Information Technology
Maharaja Agrasen Inst. of Technology
Delhi, India
ashishupadhyay9661@gmail.com

Yash Rajput
Dept. of Information Technology
Maharaja Agrasen Inst. of Technology
Delhi, India
yashrajput.ro45@gmail.com

Sahil Rao
Dept. of Information Technology
Maharaja Agrasen Inst. of Technology
Delhi, India
srao27045@gmail.com

Dr. Seema Kalonia
Dept. of Information Technology
Maharaja Agrasen Inst. of Technology
Delhi, India

***Abstract*—Schedule overruns remain one of the most common failures in construction projects, yet progress on many sites is still tracked in spreadsheets and photo folders that are reviewed separately and irregularly. Machine-learning delay predictors typically need large historical datasets that individual firms do not have, and their outputs are hard for site managers to audit. This paper presents ConstructTrack, a web-based system that (i) centralizes planned-versus-actual progress records and site photographs, (ii) computes a deterministic, fully documented schedule-risk score and projected completion date from a project's own progress velocity, and (iii) uses a YOLO11 object detector strictly as supporting visual evidence that never enters the risk computation. The scoring model combines four interpretable factors with fixed weights and makes data sufficiency explicit: no prediction is issued from fewer than two observations. On structured scenarios the framework places on-plan, ahead, slipping and stalled projects in distinct risk bands, rating a slipping project HIGH with a projected 70-day overrun where a deviation-only baseline reports MEDIUM. Results are exactly reproducible, and under 10,000 random ±30% weight perturbations the scenario ranking is preserved in every case. The system is implemented with Next.js, PostgreSQL and FastAPI and runs on free cloud tiers.**

***Index Terms*—construction progress monitoring, delay prediction, schedule risk, explainable decision support, YOLO11, computer vision.**

I. Introduction

Construction projects frequently finish late, and the causes of delay are well documented: poor planning and scheduling, slow decision-making, contractor capacity and payment problems [1], [2]. A common contributing factor is that progress information reaches decision-makers late. On many small and medium projects, planned and actual percentages are kept in spreadsheets, site photographs accumulate in chat groups or shared folders, and stakeholder reports are assembled by hand. Early signs of slippage, such as a pace too slow to finish on time while the current variance still looks small, are therefore easy to miss.

Two research directions address this problem. Vision-based progress monitoring uses site imagery, point clouds or BIM models to estimate physical progress [3], [4]. Data-driven delay prediction trains machine-learning models on historical project records [5]. Both carry adoption costs for a typical firm: the first needs registered imagery or BIM, and the second needs many completed projects with labelled outcomes. In both cases the output is difficult for a site manager to verify.

This paper takes a different position. A useful first layer of decision support can be built from data every project already has, namely planned and actual progress at known dates, provided that the computation is transparent, reproducible and honest about how much evidence supports it. Computer vision is included, but only as evidence displayed next to the numbers, never as a hidden input to them. The contributions are:

- A deterministic schedule-risk model with four interpretable factors and published weights, plus a projected completion date and delay derived from real elapsed-time progress velocity.
- An explicit data-sufficiency policy that withholds predictions when evidence is inadequate.
- An architecture that separates computer-vision evidence from schedule metrics by design.
- An open, deployable implementation with automated reporting, evaluated against a deviation-only baseline and tested for sensitivity to its weights.

II. Related Work

A. Vision-based progress monitoring

Golparvar-Fard et al. introduced D4AR, which registers site photographs against 4D BIM to compare as-built and as-planned states [3]. Kopsida et al. reviewed automated monitoring methods and noted that most require registered imagery, laser scans or BIM and remain sensitive to occlusion [4]. Deep detectors have been applied to construction scenes for workers, equipment and protective equipment [6], [7], building on single-stage detectors such as YOLO [8]. Detectors pretrained on general datasets such as COCO [9] recognize people and vehicles but not construction states such as "slab poured", so their outputs cannot be used directly as a progress measure without domain training.

B. Delay and schedule-risk prediction

Earned-value management and its schedule extension, earned schedule [10], forecast completion from planned and earned value but require cost-loaded schedules. Machine-learning approaches such as those compared by Gondia et al. [5] predict delay risk from project attributes but depend on sizeable labelled datasets. For high-stakes decisions, Rudin argues for inherently interpretable models over post-hoc explanations of black boxes [11]. Table I positions the proposed framework against these families.

TABLE I. COMPARISON WITH EXISTING APPROACHES

Approach	Inputs needed	Auditable	Abstains
Vision + BIM [3], [4]	BIM, registered imagery	Partly	No
ML delay models [5]	Labelled project history	Low	No
Earned schedule [10]	Cost-loaded schedule	Yes	No
Proposed	Planned/actual % and dates	**Yes**	**Yes**

III. METHODOLOGY

Let a project have planned start t_0 and expected end T, and progress records $\{(t_i, P_i, A_i)\}$, where P_i and A_i are planned and actual completion (%) on date t_i. Records with a recorded A_i are *usable*; let n be their number and index them chronologically from 1 to n.

A. Variance and velocity

The current schedule variance is the latest deviation

$$\Delta = A_n - P_n \tag{1}$$

Observed velocity is measured over real elapsed calendar time, so irregular reporting intervals do not distort it:

$$v = (A_n - A_1) / (t_n - t_1) \tag{2}$$

The velocity required to finish on time from the latest record is

$$v* = (100 - A_n) / (T - t_n) \tag{3}$$

and the velocity ratio is $\rho = v / v*$. Both are left undefined, never set to zero, when $T \le t_n$ or $t_n = t_1$.

B. Completion projection

If $v > 0$, the estimated completion date is $\hat{t} = t_n + (100 - A_n) / v$, and the predicted delay is

$$\delta = max(0, \hat{t} - T) \tag{4}$$

If $v \le 0$ or v is unknown, no completion date is produced, since a stalled project has no finite completion date at its current rate.

C. Schedule-risk score

Four factor scores in [0, 100] are computed from fixed thresholds (Table II): S_V maps Δ, S_ρ maps ρ, S_D scales δ relative to the planned duration $T - t_0$ (saturating at 30%), and S_T reflects the change in deviation across the last three usable records (20 improving, 50 stable, 80 worsening). The risk score is

$$R = 0.30\,S_V + 0.30\,S_\rho + 0.25\,S_D + 0.15\,S_T \tag{5}$$

rounded to an integer and banded as LOW (0–24), MODERATE (25–49), HIGH (50–74) or CRITICAL (75–100). A separate indicator, $H = 0.40\,S_V + 0.40\,S_\rho + 0.20\,S_T$, is reported as an "estimated delay probability" and is explicitly labelled a heuristic, not a calibrated probability.

TABLE II. FACTOR SCORING RULES AND WEIGHTS

Factor	Weight	Score mapping
Variance Δ	0.30	$\ge 0 \to 10$; $[-5,0) \to 40$; $[-15,-5) \to 65$; $< -15 \to 90$
Velocity ratio ρ	0.30	$\ge 1 \to 10$; $[0.8,1) \to 35$; $[0.5,0.8) \to 65$; $< 0.5 \to 90$
Predicted delay δ	0.25	$\delta/(T-t_0)$ scaled; $\ge 30\% \to 100$; early $\to 5$
Trend	0.15	Last 3 records: 20 / 50 / 80

D. Data sufficiency

Evidence strength is reported categorically rather than as a pseudo-statistical confidence: $n \le 1$ is INSUFFICIENT and no score, date or delay is computed; $n = 2$ is LIMITED; $3 \le n \le 4$ is ADEQUATE; $n \ge 5$ is STRONG. The trend factor requires $n \ge 3$. Algorithm 1 summarizes the procedure.

Algorithm 1 Schedule-risk assessment

1: **Input:** t_0, T, records $\{(t_i, P_i, A_i)\}$
2: $U \leftarrow$ usable records sorted by date; $n \leftarrow |U|$
3: **if** $n < 2$ **then return** INSUFFICIENT (no prediction)
4: Compute Δ, v, $v*$, ρ by Eqs. (1)–(3)
5: **if** $v > 0$ **then** $\hat{t}$, δ by Eq. (4) **else** $\hat{t} \leftarrow$ none
6: S_V, S_ρ, S_D, $S_T \leftarrow$ factor scores (Table II)
7: $R \leftarrow$ Eq. (5); band $\leftarrow$ threshold(R)
8: **return** R, band, $\hat{t}$, δ, factor explanations

E. Computer vision as supporting evidence

Uploaded site images are analyzed by a COCO-pretrained YOLO11n detector [12] (confidence 0.25, IoU 0.45, longest side ≤ 1280 px). Detections, per-class counts, average confidence and an annotated image are stored and shown in the interface and the report, with a factual note on how detection counts changed between visits. None of these values enters Eqs. (1)–(5), because a general-purpose detector cannot measure construction completion.

IV. SYSTEM DESIGN AND IMPLEMENTATION

ConstructTrack consists of two services (Fig. 1). A Next.js 15 application provides the user interface, REST routes with schema validation, a PostgreSQL database accessed through Prisma, and private object storage served through short-lived signed URLs. A FastAPI service performs size-capped image download with format sniffing, OpenCV preprocessing and YOLO11 inference; it receives only a signed URL and holds no database or storage credentials. The risk engine is a set of pure functions shared by the dashboard, the API and the report generator, so all three always show identical numbers. A report is generated from a single project identifier, stored as a JSON snapshot, and rendered as both a web page and a server-side PDF. The system is deployed on free tiers: Vercel for the web

application, Supabase for the database and storage, while the detector is containerised with Docker and self-hosted, protected by a shared API key. All analysis results are stored centrally, so they remain available in the deployed application at any time.

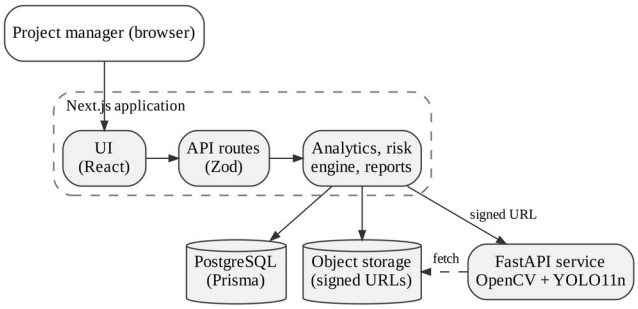


Fig. 1. System architecture.

V. Evaluation

The evaluation checks behavioral correctness, discrimination between project conditions, reproducibility and robustness to the chosen weights. It does not measure predictive accuracy, which would require outcome data (Section VI). Six scenarios were constructed with planned curves consistent with their durations and records at four-week intervals: a slipping office building (40-week plan), an on-plan residential tower (60 weeks), an ahead-of-plan retail complex (24 weeks), a stalled metro annex (36 weeks), and two edge cases. The baseline is a deviation-only classifier with thresholds of −3, −10 and −20 points.

A. Discrimination and baseline comparison

Table III. Engine Output versus Deviation-Only Baseline

Scenario	n	Δ	ρ	δ (days)	R (band)	Baseline
Office, slipping	5	−10	0.67	70	72 (HIGH)	MEDIUM
Tower, on plan	4	−1	1.00	2	23 (LOW)	LOW
Retail, ahead	3	+6	1.33	0†	10 (LOW)	LOW
Annex, stalled	4	−21	0.30	321	91 (CRIT.)	CRITICAL
Single record	1	−2	—	—	withheld	LOW
Exact pace	2	0	1.00	0	15 (LOW)	LOW

†*Projected to finish 21 days early.*

Table III and Fig. 2 show that the score orders the scenarios as expected and places them in distinct bands. The slipping office illustrates the value of combining factors: its deviation of −10 points places it only in the baseline's MEDIUM class, but its observed pace of 2.0 %/week is two-thirds of the 3.0 %/week required and its deviation has worsened at every observation. The framework projects a 70-day overrun and rates it HIGH, flagging the project earlier than variance alone. With a single record, the framework withholds a prediction while the baseline still outputs a label from one data point.

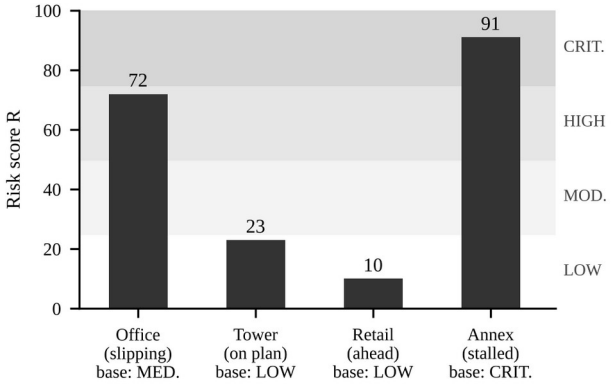


Fig. 2. Risk score R per scenario with risk bands and baseline class.

B. Reproducibility and weight sensitivity

Running the engine 1,000 times on identical input gave identical outputs. Because the weights are design choices, we tested their influence in two ways (Table IV). Scaling each weight by ±20% one at a time, with renormalization, changed no band in any of the 32 cases. Drawing 10,000 random weight vectors, each weight scaled independently by a factor in [0.7, 1.3], shifted scores by at most 5 points and preserved the ranking of the four projects in every draw. Band changes occurred in 4.6% of evaluations, all for the on-plan tower, whose score of 23 lies just below the LOW/MODERATE boundary. The engine's conclusions are therefore stable except near band edges, where the reported factor explanations matter most.

Table IV. Sensitivity to Weight Perturbation

Test	Result
±20% one-at-a-time (32 cases)	0 band changes
Random ±30%, 10,000 draws	Max score shift 5 points
Ranking of 4 projects preserved	10,000 / 10,000
Band changes (all near a boundary)	4.6% (tower only)

The implementation is covered by 61 unit tests for analytics, prediction and reporting, and 32 tests for the vision service, including an end-to-end inference test.

VI. Discussion and Limitations

The weights and thresholds are reasoned design choices and have not been fitted to outcome data, so the evaluation demonstrates consistency, interpretability and robustness rather than forecast accuracy. Velocity is a two-point linear rate and does not model learning curves, weather or resource changes. The detector is general-purpose and has not been benchmarked on labelled construction imagery. These are staged limitations rather than permanent ones: the same data model can collect outcome labels as projects close, allowing the weights to be calibrated or replaced by a fitted model, and the detector sits behind an adapter interface so a construction-specific model can be substituted without changing the scoring layer.

VII. Conclusion and Future Work

We presented an explainable framework and working system for construction schedule-risk assessment that relies only on data projects already collect, makes evidence sufficiency explicit, and keeps computer-vision output separate from schedule metrics. In scenario evaluation it distinguished project conditions that a deviation-only rule merged, remained exactly reproducible, and preserved its ranking under substantial weight perturbation. Future work includes calibrating weights against completed-project outcomes, a field study with site engineers, construction-specific detection models, and automated alerts when projects enter higher risk bands.

Acknowledgment

The authors thank Dr. Seema Kalonia, Department of Information Technology, Maharaja Agrasen Institute of Technology, for her guidance throughout this work.

References

[1] S. A. Assaf and S. Al-Hejji, "Causes of delay in large construction projects," *Int. J. Project Manage.*, vol. 24, no. 4, pp. 349–357, 2006.

[2] M. Sambasivan and Y. W. Soon, "Causes and effects of delays in Malaysian construction industry," *Int. J. Project Manage.*, vol. 25, no. 5, pp. 517–526, 2007.

[3] M. Golparvar-Fard, F. Peña-Mora, and S. Savarese, "D4AR – A 4-dimensional augmented reality model for automating construction progress monitoring data collection, processing and communication," *J. Inf. Technol. Constr.*, vol. 14, pp. 129–153, 2009.

[4] A. Kopsida, I. Brilakis, and P. A. Vela, "A review of automated construction progress monitoring and inspection methods," in *Proc. 32nd CIB W78 Conf.*, Eindhoven, 2015.

[5] A. Gondia, A. Siam, W. El-Dakhakhni, and A. H. Nassar, "Machine learning algorithms for construction projects delay risk prediction," *J. Constr. Eng. Manage.*, vol. 146, no. 1, 04019085, 2020.

[6] Q. Fang *et al.*, "Detecting non-hardhat-use by a deep learning method from far-field surveillance videos," *Autom. Constr.*, vol. 85, pp. 1–9, 2018.

[7] H. Son, H. Choi, H. Seong, and C. Kim, "Detection of construction workers under varying poses and changing background in image sequences via very deep residual networks," *Autom. Constr.*, vol. 99, pp. 27–38, 2019.

[8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779–788.

[9] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proc. ECCV*, 2014, pp. 740–755.

[10] W. Lipke, "Schedule is different," *The Measurable News*, pp. 31–34, Mar. 2003.

[11] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Mach. Intell.*, vol. 1, pp. 206–215, 2019.

[12] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics